Maastricht University

Computer Vision Assignment

Semester 2 2024-2025

Assessing Parking Behavior with Deep Learning: Identifying Vehicles in Non-Permitted Areas

Group Members:

Filippo Molino	i6391621
Emre Argın	i6405975

Department of Advanced Computing Sciences

Contents

1 Abstract **2**

2 Introduction and Related Work **2**
2.1 Problem Statement 2
2.2 Objectives 2
2.3 Related Work 2

3 Dataset and Pipeline **3**
3.1 Dataset 3
3.2 Pipeline 3

4 Vehicles Detection **4**
4.1 2D Bounding Boxes Detection 4
4.2 Depth Estimation 4
4.3 Bounding Boxes Post Processing 5
4.4 Detection Evaluation 6

5 Segmentation **6**
5.1 Semantic Segmentation 6
5.1.1 Data Preprocessing and Augmentation . 7
5.1.2 Class Imbalance Mitigation 7
5.1.3 Loss Functions 7
5.1.4 Training Procedure 7
5.1.5 Performance Evaluation 8
5.1.6 Checkpointing and Model Deployment . 8

6 Illegal Parking Marking **9**

7 Conclusions **10**

8 Contributions **10**

1 Abstract

This project develops an automated system for detecting parking violations using monocular images. By applying deep learning techniques for both object detection and semantic segmentation, the system successfully identifies vehicles, determines their parking status, and assesses whether they are within designated parking areas. Despite the challenges of working with monocular data, the system demonstrates significant potential for efficient and scalable real-world deployment in urban environments. The results highlight the feasibility of utilizing computer vision for effective parking enforcement.

2 Introduction and Related Work

2.1 Problem Statement

Parking violations are a persistent challenge in many urban environments. Vehicles parked outside of designated areas can interfere with traffic, disturb pedestrians and cyclists, and contribute to overall disorder in daily commutes. Traditional methods of enforcement—typically involving officers manually patrolling streets—are labor-intensive, inefficient, and difficult to scale in rapidly growing cities. However, recent advancements in camera technology and computer vision present a promising opportunity to automate aspects of parking enforcement, making the process more efficient, consistent, and responsive.

The aim of this project is to develop an automatic system capable of detecting when vehicles are parked in unauthorized locations by experimenting with different deep learning techniques. One possible application of this system could be to function as part of an autonomous patrol solution, for example, by integrating it into small robotic vehicles that can navigate neighborhoods and monitor parking compliance.

For the scope of our work, we decided to focus exclusively on monocular data. This means that all results are derived from a single image captured from a car. This choice was made to ensure a more flexible and straightforward implementation, without the need for dual-camera setups. Additionally, it presents a challenge that we aimed to address by exploring alternative methods, differing from those in the literature that typically rely on stereo imagery.

2.2 Objectives

For such a system to be effective, it must be capable of accomplishing several key tasks:

1. Identify the presence of vehicles
2. Determine whether the detected vehicle is parked or moving (future work)
3. Accurately detect designated parking spaces
4. Assess whether a vehicle is situated within a valid parking area

2.3 Related Work

Detecting illegal parking has been explored in the literature on some occasions. Xie et al. (2017) [9] developed a system based on the Single Shot MultiBox Detector (SSD) algorithm to detect illegally parked vehicles in real-time. By optimizing SSD and incorporating movement analysis, the system achieved 99% accuracy and operated at 25 frames per second, demonstrating strong robustness in complex environments.

More recently, De Luelmo et al. (2025) [4] introduced a hybrid approach that integrates deep learning with rule-based methods to detect parking spaces using a single front-facing camera. Their system employs Monodepth2 for depth estimation and PSPNet for road segmentation, enabling effective parking space detection without the need for specialized hardware or extensive calibration.

3 Dataset and Pipeline

3.1 Dataset

Cityscapes dataset [3] is a large-scale benchmark dataset designed for semantic urban scene understanding. It consists of high-resolution images (2048×1024 pixels) captured from 50 different cities across Germany. The dataset focuses on semantic segmentation, instance segmentation, and object detection in street scenes. It includes 5000 annotated images with fine annotations and 20000 images with coarse annotations. The main groups and classes are summarized in the following table:

Table 1: Cityscapes Dataset Class Groupings

Group	**Classes**
Flat	road, sidewalk, parking, rail track
Human	person, rider
Vehicle	car, truck, bus, on rails, motorcycle, bicycle, caravan, trailer
Construction	building, wall, fence, guard rail, bridge, tunnel
Object	pole, pole group, traffic sign, traffic light
Nature	vegetation, terrain
Sky	sky
Void	ground, dynamic, static

3.2 Pipeline

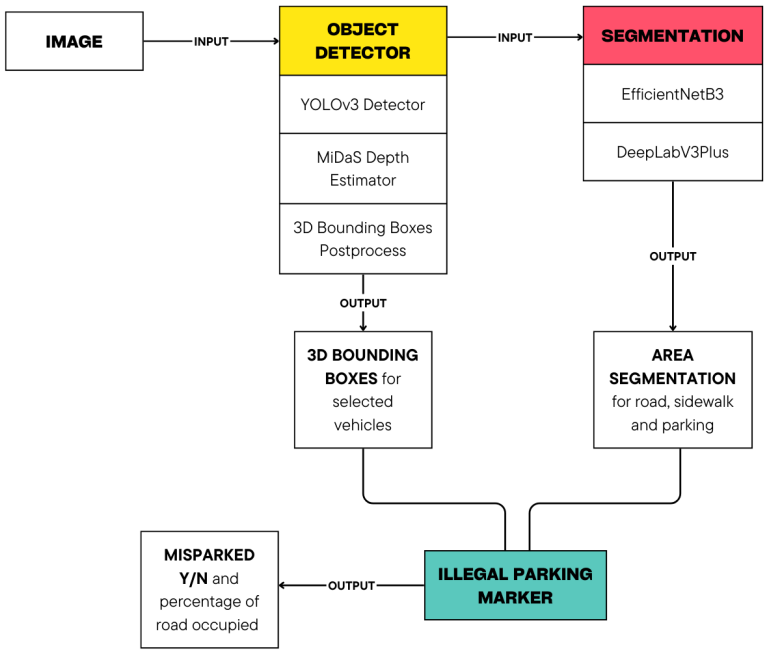


Figure 1: Pipeline Scheme for the whole process

Since our project involves two distinct computer vision tasks, object detection and segmentation, we opted to implement a pre-2020 model for the detection component and a more modern architecture for the segmentation task. After obtaining the 3D bounding boxes and road segmentation results, we combined these outputs to determine whether a vehicle was parked in an illegal location. The three modules illustrated in the pipeline in Figure 1 are further detailed in Chapters 4, 5, and 6.

4 Vehicles Detection

4.1 2D Bounding Boxes Detection

For the detection task, we decided to rely on the **YOLO model** (You Only Look Once), one of the most efficient models of the early deep learning era (2015-2019). YOLO object detector was first introduced by Redmon et al. in 2015 [7] and represents a major innovation compared to R-CNN models in terms of speed and simplicity. Unlike predecessors, YOLO performs object detection in a single neural network pass by framing it as a regression problem, simultaneously predicting bounding boxes and class probabilities directly from full images.

In our pipeline, we used YOLOv3 [8], a pre-2020 version of the model, to detect cars, motorcycles, buses, and trucks. The YOLOv3 configuration file was taken from the official Darknet GitHub repository [6]. The model was initialized with pre-trained weights [5] from the COCO dataset. Since our task involves detecting common objects such as cars, motorcycles, buses, and trucks, using transfer learning is already highly effective, since it allows the model to recognize these objects in our input images without the need to train it from scratch, saving both time and computational resources.

As shown in Figure 2, vehicles are accurately localized with 2D bounding boxes, facilitating the detection of potential parking violations.

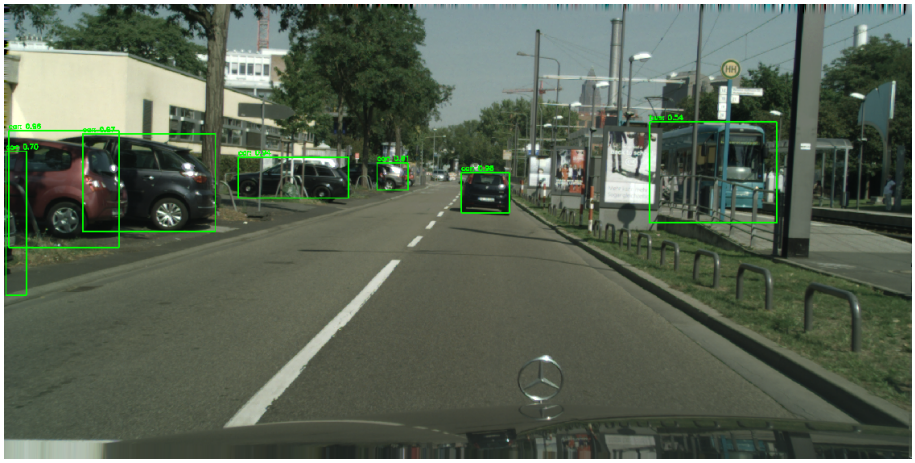


Figure 2: Example of object detection using YOLOv3. The model successfully identifies and classifies cars and the bus

4.2 Depth Estimation

For the purposes of our task, 2D bounding boxes are not sufficient. To assess whether a car is parked in a valid location, we need to understand its position relative to the ground. Therefore, it is essential to estimate 3D boundary boxes that approximate the spatial placement of the vehicle as accurately as possible.

To archive this, we combine the 2D bounding boxes produced by YOLO with depth information obtained from the **MiDaS model** [1]. MiDaS is a depth estimator that measures the approximate distance of each detected object, even though our input is a single monocular image. One sample result is shown in Figure 3.

Once we have the 2D bounding boxes from YOLO and the depth estimation from MiDaS, we use a projection method that draws a 3D bounding box around each detected vehicle. The front face of the bounding box corresponds to the 2D bounding box, while the back face is offset along the depth axis using the estimated depth. The function uses the following parameters:

- **bbox (x1, y1, x2, y2)**: The coordinates of the 2D bounding box around the vehicle, where $(x1, y1)$ is the top-left corner and $(x2, y2)$ is the bottom-right corner
- **depth**: The estimated depth of the vehicle, obtained from the MiDaS model. This value is used to shift the back face of the cube further away, giving a sense of the vehicle's position in space

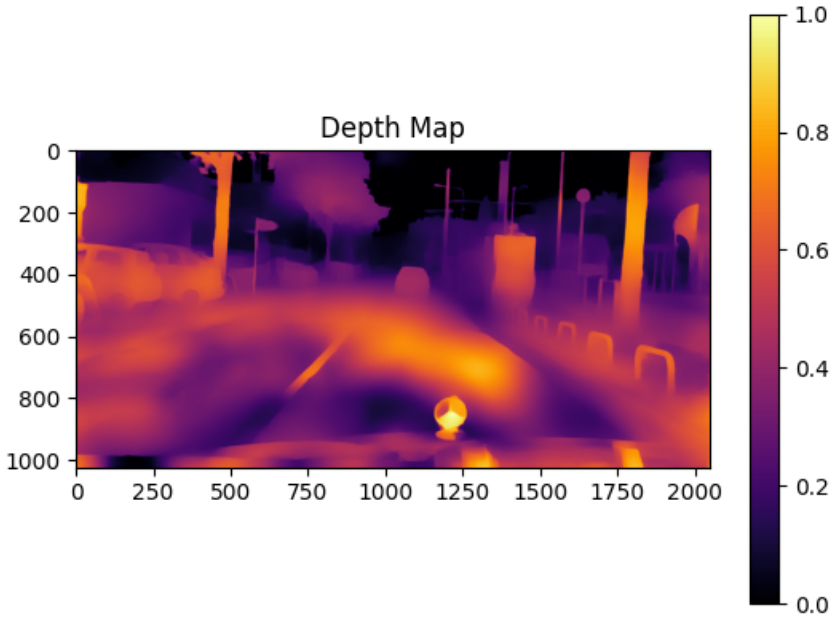


Figure 3: Depth Map of the same image as Figure 2

- **scale**: A scaling factor applied to the depth to control how much the depth affects the back face's positioning. This allows us to fine-tune the size and appearance of the 3D bounding box

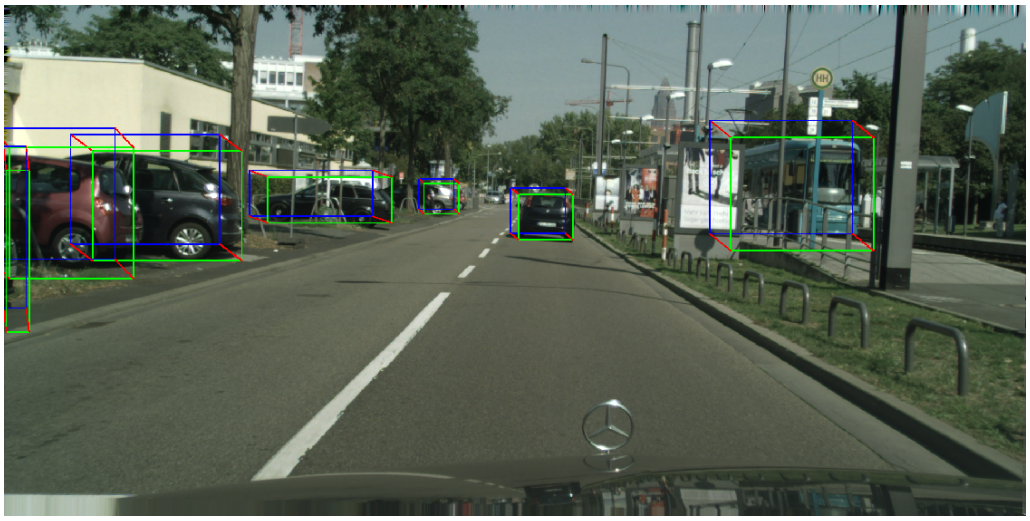


Figure 4: Raw 3D Bounding Box Estimates

4.3 Bounding Boxes Post Processing

Combining the bounding box with the depth estimation of a single point on the box does not allow for rotation, resulting in 3D boxes that often fail to capture the actual space occupied by vehicles. This limitation is clearly visible in Figure 4, where the detected boxes consistently assume the same shape. In the absence of stereo images and additional information about camera intrinsics and extrinsics, we had to refine our estimates by making certain assumptions.

Assumption 1. Given that the images are captured from a forward-facing, vehicle-mounted camera, we assume a typical perspective projection based on the horizontal position of the object

within the image. Specifically:

- For objects appearing in the **left third** of the image, we assume they are viewed from a front-left angle. As a result, we model the 3D bounding box such that the *rear face is horizontally shifted to the right* relative to the front face, simulating a natural perspective effect.
- For objects in the **right third**, we assume a front-right viewpoint, and thus the *rear face is shifted to the left* relative to the front.
- For objects located in the **central third** of the image, we assume a frontal view, with the rear face of the bounding box directly aligned behind the front face, indicating no significant horizontal offset.

Assumption 2. Since 2D bounding boxes typically represent the outermost boundaries of an object as projected onto the image plane, they often include background regions or occlusions. To avoid overestimating the size of the object in 3D space, we apply a scaling strategy: we take only the **central 80%** of the bounding box along both width and height. This adjustment reduces the likelihood of generating excessively large 3D boxes and helps to better approximate the true physical dimensions of the object.

These spatial assumptions enables us to approximate object orientation in 3D space despite the lack of stereo imagery or detailed camera calibration. Figure 5 shows the post-processed bounding boxes.

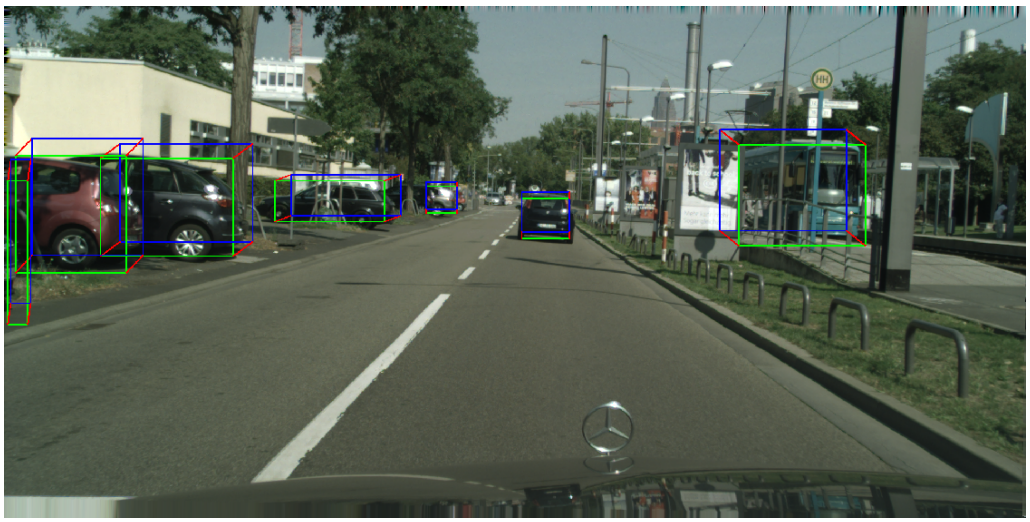


Figure 5: Refined 3D Bounding Box Estimates: 2D shrinking, spatial orientation

4.4 Detection Evaluation

This approach provides a step-by-step estimation of 3D bounding boxes around vehicles. Its major strength lies in the very low data requirements: it relies solely on monocular data, without integrating any additional information about the camera's intrinsic or extrinsic parameters, making the implementation highly flexible. The runtime is indeed very fast, requiring approximately 10 seconds. Of course, this comes at the cost of a higher degree of approximation, but it still offers a good representation of the 3D space occupied by the detected vehicles.

Additional qualitative analysis on different examples will be presented in Chapter 6.

5 Segmentation

5.1 Semantic Segmentation

For the segmentation task, we adopted the **DeepLabV3+** model, a state-of-the-art semantic segmentation architecture introduced by Chen et al. [2]. Unlike earlier models, DeepLabV3+ efficiently combines atrous convolution with spatial pyramid pooling and a lightweight decoder module, enchancing boundary delineation and pixel-level accuracy.

Our implementation utilizes DeepLabV3+ with an EfficientNet-B3 backbone, pre-trained on ImageNet. The Cityscapes dataset [3], with its comprehensive urban street-level scenes, serves as the basis for training and evaluation. To align closely with our parking violation detection task, the original Cityscapes classes were remapped into four distinct semantic categories:

- **0**: Background
- **1**: Road
- **2**: Sidewalk
- **3**: Parking zones

5.1.1 Data Preprocessing and Augmentation

To enhance robustness, our pipeline applies extensive data augmentation during training:

- **Random Scaling** (factor range 0.5 to 2.0) simulates varied camera distances.
- **Random Cropping** (512×1024) and **Horizontal Flipping** ensure spatial variability.
- **Color Jittering** (±30% in brightness, contrast, saturation) simulates diverse lighting conditions.
- **Normalization** to ImageNet mean and standard deviation.

Validation images are resized and normalized without further augmentations to maintain evaluation consistency.

5.1.2 Class Imbalance Mitigation

Due to the scarcity of "parking" class pixels, we employed weighted sampling. Images containing parking areas received a sampling weight five times higher than images without, addressing the inherent dataset imbalance and improving model sensitivity toward parking zones.

5.1.3 Loss Functions

We adopted a composite loss function tailored for our segmentation objectives:

- **Weighted Cross-Entropy Loss**, assigning class weights [1.0, 1.0, 1.0, 4.0] to increase the impact of the parking class.
- **Dice Loss**, to explicitly optimize region overlap between predicted and true masks.

The final training loss is the sum of these two components, effectively balancing accuracy across all classes.

5.1.4 Training Procedure

We trained the DeepLabV3+ model using mixed precision (AMP) over 20 epochs, structured in two main phases:

1. **Epochs 1–5**: The EfficientNet encoder is frozen, allowing only the decoder to update parameters, using an initial learning rate of 1×10^{-3}.
2. **Epochs 6–20**: The encoder is unfrozen, and the learning rate is reduced to 1×10^{-4}, with a step learning rate scheduler reducing the learning rate every 10 epochs.

The AdamW optimizer with batch size 4 was employed throughout the training. The model's segment predictions after the training can be seen in Figure 6 and Figure 7, as for demonstration purposes.

Figure 6: Segmented roads, sidewalks, and parking lots in Frankfurt

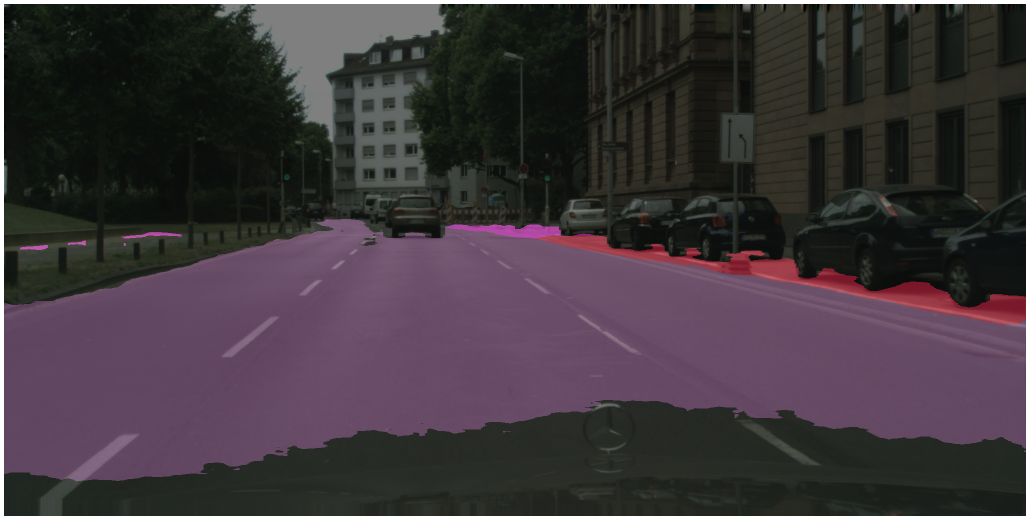


Figure 7: Segmented roads, sidewalks, and parking lots in Mainz

5.1.5 Performance Evaluation

Segmentation performance was quantitatively assessed after each epoch using Mean Intersection-over-Union (mIoU). The IoU metric evaluates pixel-level accuracy between predicted and ground-truth segmentations, with background class excluded to compute mean IoU (mIoU). This ensured precise tracking of segmentation accuracy improvements across training epochs. After 30 epochs, the loss of the model was computed as 0.36, and the mIoU was calculated as 0.61. Due to scarcity of parking lot labels, and having relatively small training instance in finely annotated dataset (consisting of 3750 training instances), the segmentation model struggled to achieve higher accuracy and it hindered the performance on mIoU.

5.1.6 Checkpointing and Model Deployment

Training progress and optimal checkpoints were preserved systematically:

- Epoch-wise checkpoints were stored, and the best-performing model based on validation mIoU was separately saved.
- Post-training, the final model underwent dynamic quantization, significantly reducing model size and accelerating inference on CPU platforms.

This segmentation workflow allows a robust, efficient approach to distinguish roads, sidewalks, and parking zones, essential for detection and monitoring of parking violations. To improve the model's

effectiveness as well as the overall accuracy, future work should revolve around incorporating more fine annotated training instance into the model.

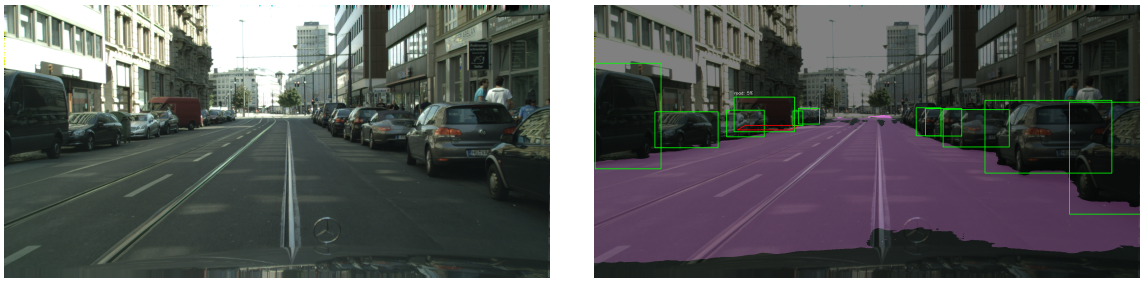
6 Illegal Parking Marking

After obtaining the 3D bounding boxes and the road segmentation from the first two modules, the final task is to determine whether a car is parked illegally, i.e. outside designated parking areas. Our approach is as follows.

For each vehicle, we consider only the bottom face of its 3D bounding box, which serves as an estimate of the area the vehicle occupies on the ground. To determine parking legality, we compute the intersection between the bottom surface of each vehicle's 3D bounding box and the segmented road surface map. This results in a set of surface class labels that the vehicle overlaps with. If the overlap involves only a single class (e.g., only road, only sidewalk, or only parking lot), the vehicle is assumed to be correctly parked. Likewise, if the overlap includes both sidewalk and parking lot (two non-road areas) we do not flag it. However, we consider a vehicle to be potentially parked illegally if its footprint overlaps with both road and sidewalk, or road and parking lot, or all three classes (road, sidewalk, and parking lot). These specific combinations suggest that the vehicle is partially occupying a non-parking area, such as a driving lane or pedestrian path. Only vehicles meeting these illegal overlap conditions are visualized for further inspection.

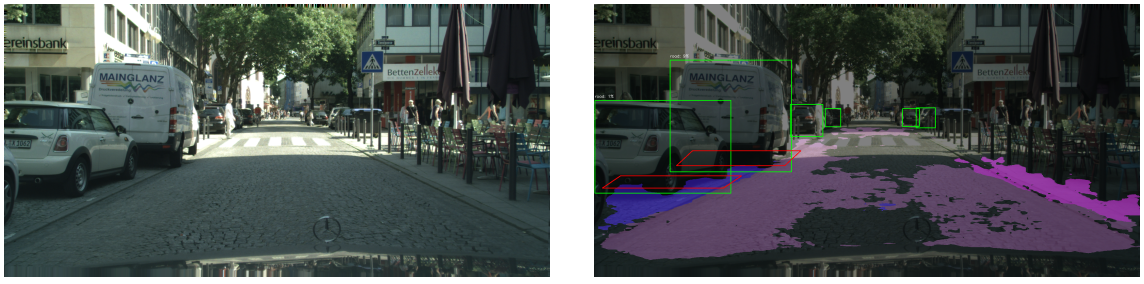
We then apply thresholding on these distributions to identify whether a car is improperly parked based on how much of its footprint lies outside designated parking areas. Example outputs from the illegal park detection pipeline can be seen in Figure 8, and Figure 9.

Note: So far, we have tested our model on individual frames. Therefore, it is not possible to distinguish moving cars from non-moving ones. To achieve this, a tracking module would be necessary to verify whether the detected cars are moving or stationary, and to only consider the parked ones. This is a possible future improvement for our project.



(a) Original Input Image (b) Detection Output

Figure 8: Side-by-side comparison of the input image and output from the illegal parking detection pipeline.



(a) Original Input Image (b) Detection Output

Figure 9: Side-by-side comparison of the input image and output from the illegal parking detection pipeline.

7 Conclusions

Working on both detection and segmentation tasks allowed us to experiment with a variety of computer vision techniques and gain a deeper understanding of their functioning, advantages, and limitations. For the detection task in particular, we had to adapt our approach to the constraint of using only monocular images without any information about the camera's intrinsic or extrinsic parameters. This led to a more approximate modeling of the scene, but it also enabled a lightweight and efficient estimation pipeline. Rather than relying on an off-the-shelf model, we were required to reason about assumptions, trade-offs, and the combination of multiple components to build a workable solution.

For the segmentation task, we trained a modern computer vision algorithm to segment sidewalks, parking areas, and roads. However, due to the relatively small size of the available dataset, the segmentation accuracy was not optimal. This experience underscored the importance of having a sufficiently large and diverse dataset to train robust models.

There are several areas in which this pipeline can be improved, starting with the integration of stereo data to enhance the accuracy of 3D bounding boxes. Additional improvements could include incorporating movement tracking to identify and exclude moving vehicles, implementing license plate extraction, and refining the segmentation model for better performance. Despite these potential enhancements, we believe this initial implementation serves as a solid example of how models can be trained and adapted to specific tasks, demonstrating their ability to address real-world challenges.

8 Contributions

Section	**Contributors**
Introduction	Emre, Filippo
Pipeline Overview	Emre, Filippo
Vehicle Detection	Filippo
Segmentation	Emre
Illegal Parking Marking	Emre, Filippo
Conclusions	Emre, Filippo

Table 2: Contributions of team members per section.

References

[1] Reiner Birkl, Diana Wofk, and Matthias Müller. Midas v3.1 – a model zoo for robust monocular relative depth estimation. *arXiv preprint arXiv:2307.14460*, 2023.

[2] Liang-Chieh Chen, Yukun Zhu, George Papandreou, Florian Schroff, and Hartwig Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation, 2018.

[3] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth, and Bernt Schiele. The cityscapes dataset for semantic urban scene understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[4] Susana P De Luelmo, Francisco J Garcia-Espinosa, Antonio S Montemayor, and Juan José Pantrigo. Combining deep learning methods and rule-based systems for automatic parking space detection. *Integrated Computer-Aided Engineering*, 32(1):97–108, 2025.

[5] Shivam Patel. Yolov3 pre-trained weights. https://www.kaggle.com/datasets/shivam316/yolov3-weights, 2020. Accessed: 2025-05-05.

[6] pjreddie. Yolov3 configuration file. https://github.com/pjreddie/darknet/blob/master/cfg/yolov3.cfg, 2018. Accessed: 2025-05-05.

[7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[8] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

[9] Xuemei Xie, Chenye Wang, Shu Chen, Guangming Shi, and Zhifu Zhao. Real-time illegal parking detection system based on deep learning. In *Proceedings of the 2017 international conference on deep learning technologies*, pages 23–27, 2017.

Google Colab Link : https://colab.research.google.com/drive/10YPh3Qe5Jsu1I5_QA6kBalNq-zK4ZzT4?usp=share_link